Homework 4: Bayesian Inference for Mixture Models

Baijia Xu

2026-04-02

Bayesian K-Mixture Model

We consider a Bayesian Mixture Model of unit-variance univariate Gaussians. There are:

- K mixture components, corresponding to K Gaussian distributions with means $\mu = \{\mu_1, \ldots, \mu_K\}$
- cluster assignments $c_i \in \{1, \ldots, n\}$
- observations $y_i \in \{1, \ldots, n\}$

Data generating model is

$$\begin{aligned} \mu &\sim N(0, \sigma^2) \\ c_i &\sim Categorical(1/K \ldots, 1/K) \\ y_i | c_i, \mu &\sim N(c_i^T \mu, 1) \end{aligned}$$

Problem 1: Gibbs sampler derivations and implementation

Derive a Gibb's sampler for the posterior distribution, $p(\mu, \mathbf{c}|y)$. Show the derivations and resulting Gibb's sampler updates. Specifically, derive:

- $p(c_i|y_i, \mu)$
- $p(\mu_k|y, c)$

Derivations

1. Conditional posterior of cluster assignments $p(c_i|y_i, \mu)$ By Bayes' rule, the posterior probability of observation i belonging to cluster k is:

$$p(c_i = k|y_i, \mu) \propto p(y_i|c_i = k, \mu)p(c_i = k)$$

Since the prior $p(c_i = k) = 1/K$ is constant for all k, we have:

$$p(c_i = k|y_i, \mu) \propto \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right)$$

Normalizing gives:

$$p(c_i = k|y_i, \mu) = \frac{\exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right)}{\sum_{j=1}^{K} \exp\left(-\frac{1}{2}(y_i - \mu_j)^2\right)}$$

2. Conditional posterior of cluster means $p(\mu_k|y,c)$ The posterior for μ_k depends on its prior and the observations assigned to cluster k:

$$p(\mu_k|y,c) \propto p(\mu_k) \prod_{i:c_i=k} p(y_i|c_i,\mu_k)$$

$$p(\mu_k|y,c) \propto \exp\left(-\frac{\mu_k^2}{2\sigma^2}\right) \prod_{i:c_i=k} \exp\left(-\frac{(y_i-\mu_k)^2}{2}\right)$$

This is a Normal-Normal conjugacy. Let n_k be the number of points in cluster k. The posterior is $N(\mu_k^*, \sigma_k^{2*})$ with:

$$\sigma_k^{2*} = \left(\frac{1}{\sigma^2} + n_k\right)^{-1}, \quad \mu_k^* = \sigma_k^{2*}\left(\sum_{i:c_i=k} y_i\right)$$

Implementation

Implement the Gibb's sampler in R. Your Gibb's R function should

- run the sampler
- store posterior draws
- return posterior summaries (estimates of the cluster assignments and means).

```r
#' Gibbs Sampler for Bayesian K-Mixture Model
#'
#' @param y Observations
#' @param K Number of clusters
#' @param sigma_sq Prior variance for mu
#' @param n_iter Number of iterations
#'
#' @return A list containing posterior samples and summaries
gibbs_sampler <- function(y, K, sigma_sq, n_iter) {
  n <- length(y)

  # Initialize parameters
  mu <- rnorm(K, 0, sqrt(sigma_sq))
  c <- sample(1:K, n, replace = TRUE)

  # Storage for samples
  mu_samples <- matrix(0, nrow = n_iter, ncol = K)
  c_samples <- matrix(0, nrow = n_iter, ncol = n)

  for (t in 1:n_iter) {
    # 1. Update cluster assignments c_i
    for (i in 1:n) {
      log_probs <- -0.5 * (y[i] - mu)^2
      # Use log-sum-exp trick for numerical stability if needed,
      # but here direct exponentiation is usually fine for unit variance
      probs <- exp(log_probs - max(log_probs))
      probs <- probs / sum(probs)
      c[i] <- sample(1:K, 1, prob = probs)
    }

    # 2. Update cluster means mu_k
```

```r
    for (k in 1:K) {
      y_k <- y[c == k]
      n_k <- length(y_k)

      post_var <- 1 / (1/sigma_sq + n_k)
      post_mean <- post_var * sum(y_k)

      mu[k] <- rnorm(1, post_mean, sqrt(post_var))
    }

    # Store samples
    mu_samples[t, ] <- mu
    c_samples[t, ] <- c
  }

  # Posterior summaries (using samples after 20% burn-in)
  burn_in <- floor(0.2 * n_iter)
  mu_est <- colMeans(mu_samples[-(1:burn_in), , drop = FALSE])

  # Estimate cluster assignments using the mode of samples
  get_mode <- function(x) {
    ux <- unique(x)
    ux[which.max(tabulate(match(x, ux)))]
  }
  c_est <- apply(c_samples[-(1:burn_in), , drop = FALSE], 2, get_mode)

  list(
    mu_samples = mu_samples,
    c_samples = c_samples,
    mu_est = mu_est,
    c_est = c_est
  )
}
```

Problem 2: Variational inference derivations

Assume a mean-field approximation. Derive a CAVI algorithm for the posterior distribution, $p(\mu, \mathbf{c}|y)$. Show explicit derivations and resulting updates. Implement the algorithm in R. You may refer to Blei, 2017 for details.

Derivations

We use the mean-field variational family:

$$q(\mu, \mathbf{c}) = \prod_{k=1}^{K} q(\mu_k; m_k, s_k^2) \prod_{i=1}^{n} q(c_i; \phi_i)$$

where $q(\mu_k)$ is $N(m_k, s_k^2)$ and $q(c_i)$ is $Categorical(\phi_{i,1}, \ldots, \phi_{i,K})$.

1. Update for $q(c_i)$ The optimal $q(c_i)$ is given by:

$$\log q(c_i = k) = \mathbb{E}_\mu[\log p(c_i = k) + \log p(y_i|c_i = k, \mu)] + \text{const}$$

$$\log q(c_i = k) = \mathbb{E}_\mu[\log(1/K) - \frac{1}{2}(y_i - \mu_k)^2] + \text{const}$$

$$\log q(c_i = k) \propto \mathbb{E}_\mu[-\frac{1}{2}(y_i^2 - 2y_i\mu_k + \mu_k^2)] = y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)$$

Thus, the assignment probabilities are:

$$\phi_{ik} \propto \exp\left(y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right)$$

2. Update for $q(\mu_k)$ The optimal $q(\mu_k)$ is given by:

$$\log q(\mu_k) = \mathbb{E}_{-\mu_k}[\log p(\mu_k) + \sum_{i=1}^n \log p(y_i|c_i, \mu)] + \text{const}$$

$$\log q(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^n \phi_{ik}\mathbb{E}_{c_i}[\log p(y_i|c_i = k, \mu_k)] + \text{const}$$

$$\log q(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^n \phi_{ik}[-\frac{1}{2}(y_i - \mu_k)^2] + \text{const}$$

Rearranging as a quadratic in μ_k:

$$\log q(\mu_k) = -\frac{1}{2}\mu_k^2\left(\frac{1}{\sigma^2} + \sum_{i=1}^n \phi_{ik}\right) + \mu_k\left(\sum_{i=1}^n \phi_{ik} y_i\right) + \text{const}$$

This is the log-density of $N(m_k, s_k^2)$ with:

$$s_k^2 = \left(\frac{1}{\sigma^2} + \sum_{i=1}^n \phi_{ik}\right)^{-1}, \quad m_k = s_k^2\left(\sum_{i=1}^n \phi_{ik} y_i\right)$$

Implementation

Your variational inference R function should

- monitor convergence using the ELBO
- return final estimates of the cluster assignments and means

```r
#' CAVI for Bayesian K-Mixture Model
#'
#' @param y Observations
#' @param K Number of clusters
#' @param sigma_sq Prior variance for mu
#' @param tol Tolerance for convergence
#' @param max_iter Maximum number of iterations
#'
#' @return A list containing estimated parameters and ELBO history
cavi_mixture <- function(y, K, sigma_sq, tol = 1e-6, max_iter = 100) {
  n <- length(y)

  # Initialize variational parameters
  m <- rnorm(K, mean(y), sd(y))
  s_sq <- rep(0.1, K)
```

```r
  phi <- matrix(1/K, nrow = n, ncol = K)

  elbo_history <- c()

  calc_elbo <- function(y, K, sigma_sq, m, s_sq, phi) {
    # Expected log joint: E[log p(y, c, mu)]
    e_log_p_mu <- sum(-0.5 * log(2*pi*sigma_sq) - (m^2 + s_sq) / (2 * sigma_sq))
    e_log_p_c <- n * (-log(K))

    # E[log p(y | c, mu)]
    e_log_p_y <- 0
    for (k in 1:K) {
      e_log_p_y <- e_log_p_y + sum(phi[, k] * (-0.5 * log(2*pi) - 0.5 * (y^2 - 2*y*m[k] + m[k]^2 + s_sq
    }

    # Entropy: -E[log q(c, mu)]
    ent_mu <- sum(0.5 * log(2*pi*exp(1)*s_sq))
    ent_c <- -sum(phi * log(phi + 1e-10))

    return(e_log_p_mu + e_log_p_c + e_log_p_y + ent_mu + ent_c)
  }

  for (iter in 1:max_iter) {
    # 1. Update phi
    for (k in 1:K) {
      phi[, k] <- exp(y * m[k] - 0.5 * (m[k]^2 + s_sq[k]))
    }
    phi <- phi / rowSums(phi)

    # 2. Update m and s_sq
    for (k in 1:K) {
      sum_phi <- sum(phi[, k])
      s_sq[k] <- 1 / (1/sigma_sq + sum_phi)
      m[k] <- s_sq[k] * sum(phi[, k] * y)
    }

    # Calculate ELBO
    curr_elbo <- calc_elbo(y, K, sigma_sq, m, s_sq, phi)
    elbo_history <- c(elbo_history, curr_elbo)

    # Check convergence
    if (iter > 1 && abs(curr_elbo - elbo_history[iter-1]) < tol) {
      break
    }
  }

  list(
    mu_est = m,
    phi = phi,
    c_est = apply(phi, 1, which.max),
    elbo_history = elbo_history
  )
}
```

Problem 3: Fit to data

Implement the Gibb's sampler and variational inference updates in R. Fit to the Old Faithful waiting times dataset used in the EM lecture.

Compare computation times and estimates from variational bayes to mean parameter estimates from the Gibb's sampler. For Gibb's sampler, also do diagnostic plots, fit multiple chains, comment on convergence and effective sample size.